

Document 12 — Production AI Engineering

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[11_multi_agent_systems](#) (Project 4 done)

How to Read & Practice This Document

- **What:** turning a script that works into a real service other people can run, trust, and pay for.
- **Why:** "it works on my computer" isn't a finished product — this document is the whole gap between a portfolio demo and a job-ready system.
- **When:** the moment anyone besides you needs to use what you built, or it needs to run without you watching it.
- **How to practice:** 1. Go through FastAPI's tutorial hands-on, in your own editor — this tool rewards typing along, not just reading. 2. Do the **Basic/Intermediate** endpoint exercises closed-book where you can. 3. Do **Real-world/Edge case** exercises against a real SQLite file, not a fake one. 4. Do the **Failure/Production** exercises for real: actually kill the database connection, actually build and run the Dockerfile. Don't skip to "I get the idea." 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, run `docker history` on your own image, and check no secret is baked in. If you haven't checked, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Project 4 (Doc11) works — but right now it only works when you personally type a command in your own terminal, on your own computer. Nobody else can use it. It can't run unattended. It has no memory of what happened five minutes ago unless your terminal is still open. This document is the gap between "it works on my machine" and "other people can actually use this, trust it, and it keeps working while nobody's watching."

Closing that gap is really four separate, boring-sounding problems, and this document takes them one at a time. First, something else needs to be able to *call* your agent — not by running a Python script, but over the network, the way every real app talks to every other real app. That's **FastAPI**: it

turns your agent into an HTTP endpoint with a typed, checked request and reply, the same Pydantic idea from Doc04/06, just applied one layer up at the web boundary. Second, once requests come from strangers instead of just you, you need a permanent, look-up-able record of what happened — which is what a **database** gives you that a Python list in memory never could: it survives a restart, and you can ask it "what happened at 3 AM?" after the fact. Third, your code needs to run the same way on a server that's never seen it before as it does on your laptop — that's what **Docker** solves, by packaging your app and everything it depends on into one portable, repeatable unit. And fourth, once real traffic starts hitting a real service, cost starts to matter in a way it never did in a terminal — which is where smart, careful **caching** comes in, saving real money without ever risking a wrong answer.

That's the whole story: **FastAPI** is the front door. **The database** is the memory. **Docker** is what makes it run anywhere, not just your machine. **Caching** is what keeps it affordable once real traffic shows up. And the **Build Task** below takes Project 4 — your multi-agent system — and wraps it in exactly these four things, so it stops being a script you run and becomes "Project 4 as an API": a real service, reachable over HTTP, backed by a database, and shippable in a container.

Topic tiers (full detail here, referenced from CURRICULUM.md)

- **Must-know:** FastAPI endpoints, request/response models, error handling, keeping secrets safe, structured logging, retries/timeouts, basic rate limiting.
- **Important:** database schema for conversation/run history, packaging with Docker, caching replies/embeddings.
- **Advanced:** login/permissions, basic CI/CD, queues for long-running agent work, scaling across servers.
- **Optional:** advanced cost-saving (prompt caching, picking cheaper models for easier tasks), running in multiple regions.

Core Concepts (read this first — everything you need is here)

FastAPI: request/response shapes are Pydantic again, one layer up

FastAPI routes take in typed request shapes and give back typed response shapes — the exact same Pydantic idea from Doc04/06, now applied at the HTTP boundary instead of tool arguments. A route function declares a Pydantic model as its input; FastAPI checks the incoming JSON against it *before* your function's code even runs, and automatically returns a clear 422 error if it doesn't match — you don't write that checking code yourself. **Why this matters:** it makes the API's contract self-explaining (FastAPI even builds interactive docs from your type hints), and it moves bad-input handling into the framework instead of scattering manual checks through your route code. **How error handling connects to Doc02:** a route should turn internal failures into the right HTTP status code — a bad-input problem is a 4xx (the client's fault, don't retry the same way), an unexpected

internal error is a 5xx (the server's fault) — and a 5xx reply should never show a raw error trace to the client. Log the details on the server, send a plain message to the client.

Database storage: what to save, and why start with SQLite

A database gives you something a Python list in memory can't: data that survives a restart, and that you can look up later (which run failed? what did agent X return at 3 AM?). **Why start with SQLite specifically:** it's a real, fully working database that doesn't need a separate server process running — the whole database is one file — so you can learn schema design, queries, and saving data without the extra work of running a separate database server. You can move to Postgres/MySQL later, using the same SQL skills, once you actually need more than SQLite can handle at once. **What to save for an agent system specifically:** enough to rebuild what happened in a run, after the fact — input, output, the steps in between, status, timestamps — but not secrets, and not so much raw data that storage cost becomes its own problem.

Docker: what a container actually is, and why it exists

A container packages your app together with everything it needs to run (the Python version, installed packages, system-level dependencies) into one portable unit that runs the same way no matter what's installed on the computer running it. **Why this solves a real, common problem:** "it works on my computer" almost always means the computer has some dependency, version, or setting your code silently depends on, that isn't written down anywhere. A container makes every dependency clear in a `Dockerfile`, so if it builds and runs from that file, it will run the same way on any computer with Docker installed — including a production server that's never seen your code before.

How a `Dockerfile` works, step by step: each line (`FROM` , `COPY` , `RUN` , `CMD`) creates a saved, unchanging layer, stacked on top of the one before it. `FROM python:3.11-slim` starts from a small base image with Python already installed. `COPY requirements.txt .` then `RUN pip install -r requirements.txt` installs your packages *before* copying the rest of your app's code — this order is on purpose: Docker reuses a saved layer if nothing that built it changed, so if you edit your app's code but not `requirements.txt`, the (usually slow) install step gets skipped on the next build, instead of running every time. `COPY . .` copies your app's code. `CMD [...]` says what runs when the container starts.

Why secrets never belong in the image: anything baked into a Docker image layer (by `COPY` -ing a `.env` file, or hardcoding a value in `ENV`) stays permanently inside that image — visible to anyone who can look at it (`docker history` shows what's in each layer), even if a *later* layer seems to remove it. The right pattern: secrets are passed in when you *run* the container, as environment variables (`docker run -e OPENAI_API_KEY=... myimage` , or a runtime `--env-file` that's itself kept out of git) — the image itself never contains a secret, at any layer, ever.

Multi-stage builds and image size (good to know, not required for this project): a multi-stage `Dockerfile` uses one stage to build things, and a second, smaller stage that only copies over the final result — keeping the shipped image small by leaving out build tools the running app doesn't

need. `.dockerignore` (like `.gitignore`, but for Docker) stops unneeded files (`.git/`, local venvs, `__pycache__`) from being copied into the image, keeping builds fast and images small.

docker-compose (good to know for running several services locally): once your system needs more than one container (like your API plus a real Postgres database instead of SQLite), `docker-compose.yml` describes how the containers run together — the network between them, shared storage, start-up order — so `docker compose up` starts your whole local setup with one command instead of several separate `docker run`s.

Caching: what's safe, and what isn't

Caching means saving a past result, so a repeated request doesn't redo expensive work. **What's safe to cache:** embeddings (the same text always gives the same embedding, from the same model — nothing to recompute), and predictable LLM replies (like `temperature=0` calls with the exact same input, where a small amount of randomness is fine). **What's not safe to cache:** anything that depends on live, changing information — a reply mentioning "today's date," fresh data from a tool call, or anything where an outdated cached answer would be actively wrong, not just slightly different. **Why this matters for cost specifically:** caching embeddings alone can remove a large amount of wasted spending in a RAG-heavy system, since the same documents otherwise get re-embedded needlessly — but caching the wrong thing (a reply that depends on live data) trades a small cost saving for silently giving wrong answers, which is a much worse trade.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [FastAPI documentation](#) — go through the Tutorial section start to finish.
- [Docker — Get Started](#) — the official walkthrough.
- [Python sqlite3 module docs](#) — move to [SQLAlchemy](#) once your schema outgrows plain SQL.
- [The Twelve-Factor App](#) — read it fully this time, not just the config part from Doc01.

Practice Exercises

Setup for this document's practice code: work inside `12_production_engineering/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install fastapi "uvicorn[standard]" sqlalchemy` .

How to run each exercise: save it as its own small script — `practice_basic.py` , `practice_intermediate.py` , and so on, matching the levels below — and run it directly: `python practice_basic.py` . Keep each one runnable on its own; don't chain them into one file.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — your first FastAPI route

- **What:** one FastAPI route (`GET /health`) with a typed response.
- **Why:** every real service needs a health check, and this is the smallest possible FastAPI app — get it running before anything more complex.
- **When you'll hit this for real:** the literal first route of Project 5.
- **How to code it:** `app = FastAPI()` , `@app.get("/health")` `def health() -> dict:`
`return {"status": "ok"}` , run with `uvicorn main:app --reload` , and hit it with `curl localhost:8000/health` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — wrap real logic behind a route

- **What:** a `POST /chat` route wrapping Project 1's chat code, with a Pydantic request model and proper 4xx/5xx replies.
- **Why:** this is the exact pattern — real logic behind a typed route with correct error handling — that Project 5's entire API layer is built from.
- **When you'll hit this for real:** wrapping any of your projects behind a real API, starting with Project 5.
- **How to code it:** a Pydantic `ChatRequest(BaseModel)` with a `message: str` field, a route that calls your Doc04 chat function and returns a `ChatResponse` , with a `try/except` translating internal errors to the right status code.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — persist and read back real data

- **What:** a SQLite table storing every run's input/output/timestamp, read back through a `GET /runs` route.
- **Why:** this is the exact save-and-look-up pattern Project 5's run history needs — practice it here on something small first.
- **When you'll hit this for real:** Project 5's Step 2, almost verbatim.
- **How to code it:** a SQLite table with `id` , `input` , `output` , `created_at` columns, an `INSERT` on every `POST /chat` call, and a `GET /runs` route that `SELECT` s and returns them all as JSON.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — a request that fails validation

- **What:** a request body that fails Pydantic's checking — confirm FastAPI's automatic 422 reply, and that no partial database write happens.

- **Why:** you need to know, concretely, that bad input never reaches your database — silent partial writes from bad requests are a real, hard-to-notice bug.
- **When you'll hit this for real:** any public-facing route, the moment a client sends malformed JSON.
- **How to code it:** send a request missing a required field with `curl` or `requests`, confirm you get 422 back, then check your database table row count didn't change.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — a dead connection, and a real container

- **What:** kill the database connection mid-request (or point at a bad path) and confirm a clean 500, not a raw error trace. Then write a working `Dockerfile`, build it, and run the API from the container, with secrets passed in only as environment variables.
- **Why:** both are real production requirements — an API that leaks internal errors to clients is a security problem, and an app that only runs "on your machine" isn't shippable.
- **When you'll hit this for real:** Project 5's own break-scenario tests, almost exactly.
- **How to code it:** point your database connection string at a nonexistent file/path temporarily, hit the route, confirm 500 with a generic message (check your terminal logs for the real detail). Then write the Dockerfile, `docker build`, and `docker run -e OPENAI_API_KEY=... your-image`, confirming it responds identically to the local version.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Project 4 as an API

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: wrap Project 4 (the multi-agent system) behind a FastAPI service, backed by a database, packaged with Docker.

Requirements:

- FastAPI route(s) exposing Project 4's task-running ability.
- Saved storage (SQLite to start) for run history: input, output, agent log, timestamp, status.
- Structured logging carried over from `01_python_foundations`, now tagged per-request (a request/run ID on every log line).
- A working `Dockerfile`; the app must run correctly with `docker run`, using only environment variables passed in at run time.
- Basic rate limiting on the public route (even a simple in-memory version is fine — write down its limitation).

Inputs: an HTTP POST with a task description (same shape as Project 4's terminal input, now over HTTP).

Outputs: an HTTP reply with the final result; a saved row in the database; tagged log lines.

Constraints: no secret should ever show up in a log line, an error reply, or the Docker image's layers.

Suggested files:

```
project_5_contentforge_pro_production/      (this work feeds directly into th
├─ api/
│   ├─ main.py
│   ├─ routes.py
│   └─ schemas.py
├─ db/
│   ├─ models.py
│   └─ database.py
├─ Dockerfile
├─ .dockerignore
└─ test_api.py
```

Functions/Components to build:

- `routes.py` → `POST /run-task` , `GET /runs/{id}` , `GET /health`
- `db/models.py` → a `Run` table (id, input, output, log, status, created_at)
- middleware or a dependency that adds a per-request ID into logs

Expected Behavior

- A valid task request returns a result and can be looked up afterward by ID.
- A bad request body returns 422 with a clear error, no database write.
- An internal failure (fake one) returns 500 with a plain, safe message to the client — full detail only in server-side logs.
- The containerized app behaves the same as the one you ran locally.

Test Cases

Scenario	Expected
Valid POST /run-task	200, result returned, row saved
Bad request body	422, no database write
Fake internal error	500, plain message to the client, full trace in logs only
<code>docker run</code> with a required env var missing	Container fails to start with a clear error, not a silent hang

Scenario	Expected
Two quick requests over the rate limit	Second request rejected with 429

Break-It / Debug Preview

- A database connection leak under repeated requests.
- A secret accidentally baked into a Docker image layer (check with `docker history`).
- A timeout that snowballs into a retry storm against your own API.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Scaling this to 10x traffic · what you'd cut first under a cost limit · how secrets safely move from environment to container · why "twelve-factor" config matters at this layer specifically.

Move On When

Project 4 is reachable over HTTP, saves state in a real database, and runs from a Dockerfile — not just `python main.py` . Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"